

Part A

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from google.colab import drive
drive.mount('/content/drive')

# Replace 'path/to/your/college.csv' with the actual path
college_df = pd.read_csv('/content/drive/My Drive/College.csv')

# Drop the college names
college_df = college_df.drop(college_df.columns[0], axis=1)

# Transform 'Private' column using Label Encoding
encoder = LabelEncoder()
college_df['Private'] = encoder.fit_transform(college_df['Private'])

# Define features (X) and target variable (y)
X = college_df.drop('Apps', axis=1) # Assuming 'Apps' is the target variable
y = college_df['Apps']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42) # You can adjust test_size and random_state

# Now you have:
# X_train, y_train: Training data (features and target)
# X_test, y_test: Testing data (features and target)

print("Training set size:", len(X_train))
print("Testing set size:", len(X_test))
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Training set size: 621

Testing set size: 156

Part B

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

# Create a linear regression model
```

```

model = LinearRegression()

# Fit the model on the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate the test error (mean squared error)
test_error = mean_squared_error(y_test, y_pred)

print("Test Error (Mean Squared Error):", test_error)

# R-squared
r_squared = r2_score(y_test, y_pred)
print("R-squared:", r_squared)

```

Test Error (Mean Squared Error): 1492443.3790390454

R-squared: 0.8877583168400976

Part C

```

from sklearn.linear_model import RidgeCV

# Create a Ridge regression model with cross-validation to choose lambda
(alpha)
ridge_model = RidgeCV(alphas=[0.1, 1.0, 10.0], cv=5) # You can adjust the
alphas and cv

# Fit the model on the training data
ridge_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred_ridge = ridge_model.predict(X_test)

# Calculate the test error (mean squared error) for ridge regression
test_error_ridge = mean_squared_error(y_test, y_pred_ridge)

print("Test Error (Ridge Regression, Mean Squared Error):", test_error_ridge)

# R-squared for Ridge Regression
r_squared_ridge = r2_score(y_test, y_pred_ridge)
print("R-squared (Ridge Regression):", r_squared_ridge)

```

```
# You can also print the chosen alpha (lambda) value:
print("Chosen alpha (lambda) for Ridge Regression:", ridge_model.alpha_)
```

Test Error (Ridge Regression, Mean Squared Error): 1478572.8112797

R-squared (Ridge Regression): 0.8888014759264375

Chosen alpha (lambda) for Ridge Regression: 10.0

Part D

```
from sklearn.linear_model import LassoCV

# Create a Lasso regression model with cross-validation to choose lambda
(alpha)
lasso_model = LassoCV(cv=5) # You can adjust cv

# Fit the model on the training data
lasso_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred_lasso = lasso_model.predict(X_test)

# Calculate the test error (mean squared error) for lasso regression
test_error_lasso = mean_squared_error(y_test, y_pred_lasso)

print("Test Error (Lasso Regression, Mean Squared Error):", test_error_lasso)

# R-squared for Lasso Regression
r_squared_lasso = r2_score(y_test, y_pred_lasso)
print("R-squared (Lasso Regression):", r_squared_lasso)

# Count the number of non-zero coefficients
non_zero_coefficients = sum(lasso_model.coef_ != 0)
print("Number of non-zero coefficients:", non_zero_coefficients)

# You can also print the chosen alpha (lambda) value:
print("Chosen alpha (lambda) for Lasso Regression:", lasso_model.alpha_)
```

Test Error (Lasso Regression, Mean Squared Error): 1587020.0176529174

R-squared (Lasso Regression): 0.8806455236482635

Number of non-zero coefficients: 7

Chosen alpha (lambda) for Lasso Regression: 14444.597843675856

Part E

```
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
import numpy as np

max_components = X_train.shape[1]
M_values = range(1, min(20, max_components) + 1)

kf = KFold(n_splits=10, shuffle=True, random_state=42)

cv_errors = []

for M in M_values:
    mse_list = []
    for train_idx, val_idx in kf.split(X_train):
        X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

        pca = PCA(n_components=M)
        X_tr_pca = pca.fit_transform(X_tr)
        X_val_pca = pca.transform(X_val)

        lm = LinearRegression()
        lm.fit(X_tr_pca, y_tr)

        preds = lm.predict(X_val_pca)
        mse_list.append(mean_squared_error(y_val, preds))

    cv_errors.append(np.mean(mse_list))
best_M = M_values[np.argmin(cv_errors)]
print("Best M:", best_M)

pca_final = PCA(n_components=best_M)
X_train_pca = pca_final.fit_transform(X_train)
X_test_pca = pca_final.transform(X_test)

lm_final = LinearRegression()
lm_final.fit(X_train_pca, y_train)

y_pred_pcr = lm_final.predict(X_test_pca)
pcr_test_mse = mean_squared_error(y_test, y_pred_pcr)

print("PCR Test MSE:", pcr_test_mse)
```

Best M: 17

PCR Test MSE: 1492443.3790390273